

Solving Constraint Optimization Problems Using Learning-Based Techniques

Supplementary Technical Report

Romaissa GHLIB, Abdelkader OUALI, Samir LOUDNI, Jean-luc LAMOTTE

September 23, 2026

Abstract

This supplementary technical report provides detailed experimental and implementation information for the CL-VNS framework presented in the main paper. The purpose of this document is to provide a complete description of the experimental protocol, data collection procedure, feature engineering, graph representation, model architecture, training procedure, hyperparameter configuration, validation strategy, and inference procedure.

Contents

1	Data Collection and Representation	2
1.1	Data Collection Configuration	2
1.2	CFN Graph Representation	3
1.3	Feature Engineering	4
1.3.1	Variable Node Features	4
1.3.2	Cost-Function Node Features	6
1.3.3	Edge Features	7
2	Learning Framework	7
2.1	GNN Policy Network	7
2.1.1	Message Passing	7
2.1.2	Attention Mechanism	8
2.1.3	Output Layer	8
2.2	Contrastive Learning Objective	8
2.3	Training Procedure	9
2.3.1	Learning-Rate Annealing	9
3	Inference and Experimental Configuration	10
3.1	Neighborhood Construction	10
3.1.1	Greedy Selection	10
3.1.2	Stochastic Selection	10

1 Data Collection and Representation

The proposed approach follows a three-phase experimental protocol: data collection, model training, and evaluation. First, VNS/LDS+CP is run on the training instances to collect improving and non-improving neighborhoods. The collected samples are then represented as bipartite CFN graphs and used to train the GNN with a contrastive objective. Finally, the learned policy is integrated into VNS/LDS+CP and evaluated on unseen Computational Protein Design (CPD) instances. The overall pipeline is illustrated in Fig. 1.

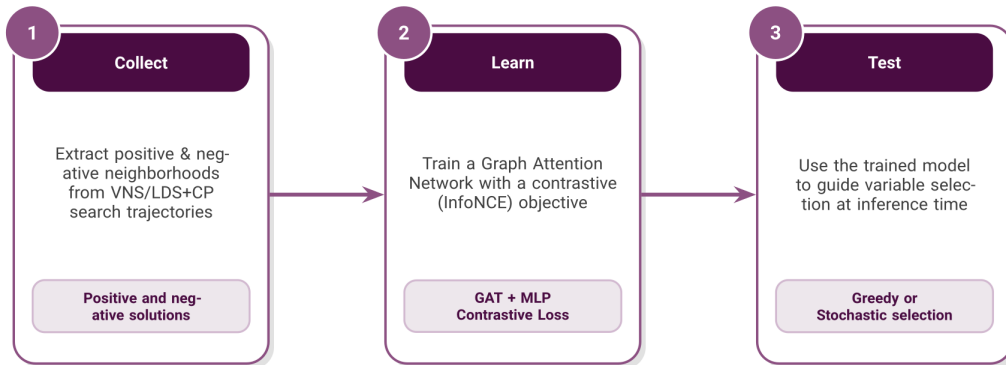


Figure 1: Overview of the CL-VNS experimental pipeline.

1.1 Data Collection Configuration

The VNS/LDS+CP algorithm is used both for collecting training data and as the underlying optimization procedure during inference.

The main parameters are summarized in Table 1.

Table 1: VNS/LDS+CP configuration used for data collection.

Parameter	Value	Description
Time limit per instance	500 s	Wall-clock time budget per run
Initial neighborhood size $k_{\min}$	4	Minimum number of relaxed variables
Maximum neighborhood size $k_{\max}$	15	Maximum number of relaxed variables
Neighborhood increment	<code>k_add1</code>	Increase neighborhood size by one
Number of runs	5	Independent runs per instance
Window size increment ℓ	Adaptive / configured	Increment used to construct cumulative windows
Positive threshold α_p	0.5	Minimum relative quality required for a positive sample
Negative threshold α_n	0.1	Maximum relative improvement allowed for a negative sample
Contrastive ratio κ	9	Target number of negatives generated per positive
Perturbation ratios R	$[0.1, 0.2, \dots, 0.9]$	Fraction of variables replaced in a positive neighborhood
Maximum repair attempts	10	Maximum attempts for each perturbation ratio
Maximum positives per window u_w	8–10	Limits the number of positive samples retained per window

All instances used for data collection are first preprocessed using the TOULBAR2 solver. The preprocessing is performed before running the VNS/LDS+CP procedure from which the training data are collected.

The main purpose of this preprocessing step is to obtain a simplified and more informative representation of the constraint network before feature extraction. Preprocessing can remove redundant information and propagate information that is already implied by the constraints, thereby providing a cleaner representation of the problem structure. This is particularly relevant for our learning approach, since the features used by the GNN are extracted from the variables, cost functions, and their relationships in the constraint network.

Consequently, performing preprocessing before data collection allows the extracted features to better reflect the effective structure of the instances explored during the data-generation process, while reducing the influence of redundant structures. The preprocessing is therefore applied only during data collection and is not part of the evaluation procedure.

The preprocessing command used for data collection is:

```
1 toulbar2 exemple.wcsp -A -p=3 -f=1 -dec -z=2 -z=instance_pretraitee.wcsp
```

1.2 CFN Graph Representation

Each CFN instance is represented as a bipartite graph

$$G = (X \cup W, E),$$

where:

- X is the set of variable nodes,

- W is the set of cost-function nodes,
- E is the set of variable–cost-function edges.

An edge

$$e = (x, w_S)$$

exists whenever variable x belongs to the scope S of cost function w_S .

This representation captures both the structural dependencies of the CFN and the relationships between variables and cost functions.

1.3 Feature Engineering

The graph representation is enriched with static and dynamic features.

All node and edge features are computed exclusively from information available at the current search iteration and current cumulative window. No information from future search iterations is used to construct the features associated with a current state. This constraint is important to avoid information leakage between the search process and the learned policy.

1.3.1 Variable Node Features

Each variable node in the bipartite constraint graph is represented by a feature vector combining structural information about the WCSP instance and information describing the current search state.

The feature vector can be written as

$$\mathbf{f}_x = [\mathbf{f}_x^{\text{static}} \parallel \mathbf{f}_x^{\text{dynamic}}],$$

where $\parallel$ denotes concatenation.

- **Static features:** The static features characterize the structural properties of a variable and its surrounding constraint network. They include domain information, structural degree, cost statistics of incident constraints, and the distribution of constraint arities.
- **Dynamic features:** Dynamic features capture the state and history of the search at the moment when a training sample is collected. They describe the current assignment, search progress, neighborhood characteristics, variable selection history, improvement history, and local cost information.

The complete list of features used for variable nodes is provided below [3 2](#).

Table 2: Static features used for variable nodes.

Feature	Description
dom_size_norm	Normalized domain size.
degree_norm	Normalized number of incident constraints.
is_binary	Indicates whether the variable has a binary domain.
avg_cost_mean_norm	Average cost statistics of incident constraints.
max_amplitude_norm	Maximum cost variation among incident constraints.
unary_min	Minimum unary cost statistic.
unary_max	Maximum unary cost statistic.
unary_mean	Mean unary cost statistic.
unary_std	Standard deviation of unary costs.
unary_range	Range of unary costs.
incident_cost_std_mean	Mean cost dispersion of incident constraints.
incident_amplitude_mean	Mean cost amplitude of incident constraints.
incident_amplitude_std	Standard deviation of cost amplitudes of incident constraints.
num_unary_constraints	Number of incident unary constraints.
num_binary_constraints	Number of incident binary constraints.
num_higher_arity_constraints	Number of incident constraints with arity greater than two.

Table 3: Dynamic features used for variable nodes.

Feature	Description
curr_val_norm	Current value assigned to the variable.
ub_norm	Current upper-bound information.
lb_norm	Current lower-bound information.
delta_ub	Recent improvement in the upper bound.
time_since_impr	Time or iteration information since the last improvement.
k_norm	Normalized size of the current VNS neighborhood.
success_rate	Overall repair success information.
freq_in_neigh	Historical frequency of the variable being selected in neighborhoods.
freq_in_winning	Historical frequency of the variable in successful neighborhoods.
freq_in_losing	Historical frequency of the variable in unsuccessful neighborhoods.
n_val_changes	Historical frequency of changes to the variable’s assigned value.
avg_gain_norm	Average improvement associated with the variable.
stagnation_per_var	Variable-specific stagnation information.
global_impr_rate	Global improvement rate of the search.
curr_unary_cost	Unary cost associated with the current value.
delta_flip	Local improvement potential associated with changing the variable’s value.
energy_grad	Sensitivity of the local cost to the variable’s value.
conflict_count	Current conflict information for the variable.
constraint_violation_score	Current local constraint-violation information.

To provide the model with short-term information about recent successful search decisions, the representation also includes features derived from the three most recent improving iterations ($\text{window_k}=3$).

For each of these three iterations, two features are stored:

Table 4: Short-term window features used for variable nodes.

Feature	Description
<code>win_mask_i</code>	Indicates whether the variable belonged to the neighborhood associated with the i -th most recent improving iteration.
<code>win_sol_i</code>	Records the value assigned to the variable in the solution obtained at the i -th most recent improving iteration.

Thus, the window representation provides six additional features, corresponding to the two features associated with each of the three most recent improving iterations. This short-term information complements the dynamic features by explicitly retaining recent successful search patterns.

1.3.2 Cost-Function Node Features

Each cost-function node $w \in W$ is represented by 18 features describing the structure, connectivity, and cost distribution of its associated cost table. Most features are static and are extracted once from the WCSP instance, whereas two features are dynamic and are recomputed for each current graph.

#	Feature	Description
0	<code>arity_norm</code>	Normalized size of the constraint scope.
1	<code>log_nb_tuples</code>	Log-scaled number of explicitly represented tuples, reducing the effect of large differences in table size.
2	<code>cost_min</code>	Minimum cost attainable by the constraint, normalized with respect to the maximum cost scale.
3	<code>cost_max</code>	Maximum explicit cost of the constraint.
4	<code>cost_mean</code>	Mean cost of the tuples in the cost table.
5	<code>cost_std</code>	Dispersion of tuple costs within the cost table.
6	<code>log_density</code>	Log-scaled density of the explicit cost table relative to the possible tuple space.
7	<code>hard_like_ratio_top</code>	Proportion of tuples having costs close to the maximum cost, indicating potentially restrictive combinations.
8	<code>scope_deg_min</code>	Minimum connectivity of the variables belonging to the constraint scope.
9	<code>scope_deg_mean</code>	Average connectivity of the variables belonging to the constraint scope.
10	<code>scope_deg_max</code>	Maximum connectivity among the variables in the constraint scope.
11	<code>cost_cv</code>	Relative heterogeneity of the tuple costs, measuring cost variability independently of the overall scale.
12	<code>default_cost</code>	Normalized cost assigned to tuples that are not explicitly listed in the cost table.
13	<code>ratio_in_neigh</code> (<i>dyn.</i>)	Proportion of variables in the constraint scope that belong to the current neighborhood.
14	<code>mean_activity</code> (<i>dyn.</i>)	Mean recent search activity of the variables belonging to the constraint scope.
15	<code>cost_amplitude</code>	Normalized range between the minimum and maximum costs of the constraint.
16	<code>zero_cost_ratio</code>	Proportion of tuples associated with zero cost.
17	<code>penalizing_ratio</code>	Proportion of tuples associated with a positive cost.

Table 5: Cost-function node features. Dynamic features are marked with *dyn.*

1.3.3 Edge Features

Each edge connects a variable to a cost-function node.

The current representation contains two edge-level features:

1. the arity of the cost function associated with the edge;
2. an indicator describing the local cost contribution of the incident variable.

The second feature is binary:

$$f_e^{(2)} = \begin{cases} 1 & \text{if the incident variable reduces the local cost,} \\ 0 & \text{otherwise.} \end{cases}$$

Thus, edge features provide localized information about the relationship between a variable and the cost functions in which it participates.

2 Learning Framework

2.1 GNN Policy Network

The policy network is based on a Graph Attention Network operating on the bipartite CFN representation.

Before message passing, variable, cost-function, and edge features are projected into a common embedding space of dimension

$$d = 64.$$

The initial embeddings are therefore:

$$h_x^{(0)} = W_x \mathbf{f}_x,$$

$$h_w^{(0)} = W_w \mathbf{f}_w,$$

$$h_e^{(0)} = W_e \mathbf{f}_e.$$

2.1.1 Message Passing

A message-passing block contains two propagation directions.

Variable-to-Cost-Function Propagation. Each cost-function node aggregates information from the variables in its scope:

$$h_w^{(1)} = \text{GAT}_{X \rightarrow W} \left(\{h_x^{(0)} : x \in S(w)\}, h_w^{(0)} \right).$$

This enables each cost-function representation to encode information about the variables participating in its scope.

Cost-Function-to-Variable Propagation. The variable representations are subsequently updated using their incident cost functions:

$$h_x^{(1)} = \text{GAT}_{W \rightarrow X} \left(\{h_w^{(1)} : w \ni x\}, h_x^{(0)} \right).$$

This allows each variable representation to incorporate information from the cost functions in which it participates.

2.1.2 Attention Mechanism

The GAT uses multi-head attention with

$$H = 8$$

attention heads.

The attention mechanism dynamically assigns different weights to neighboring nodes instead of uniformly aggregating their information.

The resulting representation captures complementary aspects of the local variable–cost–function dependencies.

2.1.3 Output Layer

After message passing, each variable embedding is passed through an MLP:

$$s_i = \sigma \left(\text{MLP}(h_{x_i}^{(L)}) \right).$$

The sigmoid function is

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

so that

$$s_i \in [0, 1].$$

The resulting value represents the predicted relevance of variable x_i for neighborhood construction.

2.2 Contrastive Learning Objective

For each cumulative search window, the policy network produces a score vector

$$\pi_\theta(S_w) \in [0, 1]^{|X|}.$$

A neighborhood is represented by a binary vector

$$a \in \{0, 1\}^{|X|},$$

where

$$a_i = \begin{cases} 1 & \text{if variable } x_i \text{ belongs to the neighborhood,} \\ 0 & \text{otherwise.} \end{cases}$$

The compatibility between a neighborhood and the learned policy is defined using

$$a^\top \pi_\theta(S_w).$$

The contrastive objective is

$$\mathcal{L}(\theta) = \sum_w \frac{1}{|D_w^+|} \sum_{a \in D_w^+} -\log \frac{\exp(a^\top \pi_\theta(S_w)/\tau)}{\sum_{a' \in D_w^- \cup \{a\}} \exp(a'^\top \pi_\theta(S_w)/\tau)}.$$

The temperature parameter τ controls the sharpness of the contrastive distribution.

2.3 Training Procedure

A batch is constructed from the data associated with one instance within one cumulative window. Samples from different cumulative windows are never mixed within the same batch. This preserves the temporal and search-contextual structure of the collected data and ensures that the positive and negative samples used for contrastive learning belong to the same cumulative window.

The model is trained using AdamW with AMSGrad enabled.

The optimization parameters are summarized in Table 6.

Table 6: GNN training configuration used for the CPD experiments.

Parameter	Value
Number of epochs	50
Batch size	1 state / batch
Learning rate	0.000132
Learning-rate annealing	Enabled
Contrastive temperature τ	0.07
Embedding dimension	64
Number of GAT layers	1
Number of attention heads	8
Dropout	0.48
Weight decay	5×10^{-4}
Optimizer	AdamW
AMSGrad	Enabled
LR decay after	20 epochs without improvement
Give-up / early stopping	50 epochs without improvement

2.3.1 Learning-Rate Annealing

When the `--anneal-lr` option is enabled, a cosine annealing schedule is used.

The learning rate is progressively reduced during training rather than remaining fixed at its initial value.

In addition, if the validation loss does not improve for the specified number of epochs, the learning rate is manually reduced by a factor of two.

The current configuration uses:

- initial learning rate: 1.32×10^{-4} ;
- LR decay patience: 20 epochs;
- decay factor: $1/2$;
- maximum training duration: 50 epochs.

The validation procedure is performed at the **instance level** rather than at the individual solution level.

This distinction is important because multiple search states and solutions can originate from the same instance.

If an instance is assigned to the validation set, all samples generated from that instance are kept in the validation set.

Consequently, samples from the same trajectory cannot simultaneously appear in the training and validation sets.

This strategy reduces temporal correlation and limits information leakage between the two sets.

The default split is:

$$80\% \text{ training, } \quad 20\% \text{ validation.}$$

Alternatively, specific instances can be reserved using the `VALID_INSTANCES` configuration.

During training, model checkpoints are saved to allow the recovery of the best-performing model.

The training procedure includes:

- saving the model after each epoch;
- tracking the validation loss;
- monitoring AUC as an additional ranking-quality indicator;
- stopping immediately if the loss becomes NaN;
- stopping after the configured number of epochs without improvement.

The trained network can additionally be exported to TorchScript for integration into the solver implementation.

3 Inference and Experimental Configuration

3.1 Neighborhood Construction

At inference time, the trained policy network receives the current CFN graph together with the dynamic search features computed from the current search state.

The network outputs one relevance score for every variable:

$$\pi_{\theta}(S)_i \in [0, 1].$$

These scores are then used to construct the next neighborhood.

3.1.1 Greedy Selection

The greedy strategy selects the k variables with the highest predicted scores:

$$N_k = \text{TopK}(\pi_{\theta}(S), k).$$

This strategy is deterministic and favors intensification.

3.1.2 Stochastic Selection

The stochastic strategy transforms the predicted scores into a probability distribution, for example using a softmax:

$$p_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}.$$

Variables are then sampled according to this distribution without replacement.

This strategy maintains a balance between exploiting variables with high predicted relevance and exploring alternative neighborhoods.

At inference time, both greedy and stochastic selection were tested. Stochastic sampling provided greater neighborhood diversity and mitigated search stagnation; therefore, it is used in the reported experiments.